Instituto Superior de Engenharia do Porto

Licenciatura em Engenharia Biomédica

Relatório Intermédio Individual

Trabalho desenvolvido – Visão Computacional

Laboratórios de Sistemas Biomédicos

Docente:

Pedro Guimarães

Autora:

Lícia Almeida 1211459

Data de realização:

fevereiro 2024

RESUMO

No âmbito da unidade curricular de Laboratórios de Sistemas Biomédicos foi proposto o desenvolvimento de um projeto inovador na área da saúde e bem-estar, de forma a aprimorar competências técnicas e conhecimentos na nossa área de estudo, preparando-nos para os desafios e exigências do mercado de trabalho. Assim, o projeto escolhido foi um sistema de monitorização do sono, integrado numa almofada que, para além de registar tempos de sono e vigília, permite ao utilizador selecionar horários para adormecer e acordar, enquanto liberta melatonina e música para facilitar o sono.
O presente relatório consiste na descrição da parte da visão computacional para a deteção de movimentos do utilizador, para determinar os tempos dedicados ao sono ou à vigília, durante a noite, de acordo com o horário escolhido pelo utilizador de forma a analisar a evolução dos padrões de evolução do sono, sendo para isto, realizado um gráfico de barras que indique o tempo do sono do utilizador.

ÍNDICE

1. INTRODUÇÃO 1
2. TRABALHO DESENVOLVIDO 1
2.1 Desenvolvimento de algoritmos 1
2.1.1 Módulo principal *detecao_mov.py* 1
2.1.2 Módulo *python_send_to_mysql.py* 4
2.1.3 Módulo *mySQL_to_python.py* 5
2.2 Criação de base de dados 5
2.3 Criação do histórico 6
2.4 Visão geral 7
3. RESULTADOS OBTIDOS 8
4. CONCLUSÃO 9
5. REFERÊNCIAS 10

1. INTRODUÇÃO

No domínio da unidade curricular de Laboratórios de Sistemas Biomédicos foi proposta a elaboração do presente relatório intermédio, para tornar possível a avaliação individual de competências, acerca do projeto em desenvolvimento, sendo neste caso, um sistema de monitorização do sono. O sistema deverá ter a capacidade de registar os tempos de sono e vigília, através de uma câmara, que serão apresentados no *website*, onde também será possível a escolha do horário em que o utilizador pretende acordar e adormecer, libertando, neste último, pequenas quantidades de melatonina e música promover o sono. A arquitetura do sistema está representada na Figura 1.

A minha contribuição para o desenvolvimento do projeto em questão, concentrou-se na área da visão computacional, que permitiu a deteção de movimentos do utilizador. A partir daí, foi possível determinar o tempo dedicado ao sono ou à vigília, de acordo com o horário pré-estabelecido pelo utilizador, e, o registo desses dados numa base de dados, para possibilitar, numa fase posterior, a sua representação gráfica através de um gráfico de barras, que permitirá visualizar a evolução dos padrões de evolução do sono, bem como, analisar a eficácia do sistema desenvolvido. Para calcular os tempos de sono e vigília, assumiu-se que o utilizador se encontraria a dormir, quando não houvesse deteção de movimento, e, por conseguinte, encontrar-se-ia acordado, quando fosse detetado movimento.

A visão computacional, no âmbito da inteligência artificial, centra-se na capacidade de computadores identificarem objetos. Este campo da IA tem em vista a criação de sistemas automatizados capazes de interpretar dados visuais, de forma similar ao processo realizado pelo ser humano, dado que, os computadores fazem a interpretação detalhada de imagens, pixel por pixel. Depois da extração das informações visuais, esses dados são processados e analisados por programas de *software* avançados permitindo o reconhecimento de padrões, a identificação de objetos e reação a estímulos visuais, aproximando-se, desta forma, da capacidade humana de interpretação visual [1].
Nesta parte do projeto, dedicada à visão computacional, foi criado um algoritmo que permitisse o cálculo dos tempos referidos anteriormente, através da deteção de movimentos do utilizador, através de uma câmara, representada na secção A da Figura 1.

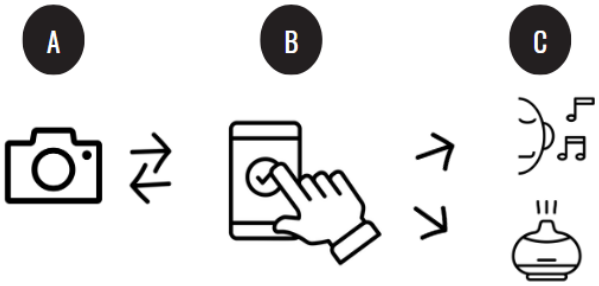


Figura 1 – Esquema da Arquitetura do Sistema.

2. TRABALHO DESENVOLVIDO

Nesta secção será apresentado o trabalho desenvolvido no que diz respeito ao meu contributo individual, isto é, a parte do desenvolvimento de algoritmos relativos à visão computacional.

2.1 Desenvolvimento de algoritmos

Para o objetivo final, foram desenvolvidos três algoritmos interligados entre si: o algoritmo principal com a função que permitiu obter os tempos de duração do sono e da vigília (*detecao_mov.py*), e os algoritmos que comunicam com a base de dados, um que envia os dados do *Python* para o *mySQL* (*python_send_to_mysql.py*) e outro que recebe os dados do *mySQL* para o *Python* (*mySQL_to_python.py*).

2.1.1 Módulo principal *detecao_mov.py*

A função *obter_tempos()* (Figura 2) permite, tal como era previsto, obter os registos dos tempos necessários à realização do projeto.

Para detetar os movimentos foi necessário implementar a biblioteca *OpenCV*, que permitiu o acesso ao módulo cv2, que possibilita o acesso à leitura de vídeos em tempo real, deteção de objetos ou partes do corpo e o seu movimento [2].

Numa fase inicial, os testes foram feitos com a câmara integrada do computador, mas numa fase posterior, foi utilizada uma câmara externa, alterando-se apenas o índice, da mesma, no algoritmo, para 1 (Figura 2).

A subfunção *cv2.createBackgroundSubtractorMOG2()* (Figura 2) faz parte da biblioteca *OpenCV* e foi utilizada para criar um objeto que implementa um algoritmo de subtração de plano de fundo, especificamente o MOG2 (*Mixture of Gaussians 2*). A subtração de plano de fundo é uma técnica que, comummente, se utiliza em visão computacional para isolar objetos em movimento num cenário, removendo o plano de fundo estático, aplicada aos *frames* de vídeos ou imagens individuais [3].

```
def obter_tempos():
    # Inicializar a câmera do PC
    cap = cv2.VideoCapture(0)  # 0 = câmara padrão

    # Configurar o background subtractor
    fgbg = cv2.createBackgroundSubtractorMOG2()
```

Figura 2 - Inicialização da câmara e configuração do *background subtractor*.

Para determinar os tempos de sono e vigília, assumiu-se que o indivíduo se encontrava acordado quando estava em movimento, ou seja, quando houvesse deteção do mesmo pelo algoritmo (Figura 3).

```
# Verificar se é o momento desejado para iniciar a detecção de movimento
if tempo_adormecer <= hora_atual <= tempo_acordar:

    # Aplicar o background subtractor para detectar movimento
    fgmask = fgbg.apply(frame)

    # Aplicar um threshold à máscara de foreground
    _, thresh = cv2.threshold(fgmask, 25, 255, cv2.THRESH_BINARY)

    # Encontrar contornos na imagem thresholded
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    # Desenhar retângulos ao redor dos objetos em movimento
    motion_detected = False
    for contour in contours:
        if cv2.contourArea(contour) > 500:                                  # Aj
            if motion_start_time is None:
                motion_start_time = hora_atual
                print("Movimento Iniciado")

            motion_detected = True
            (x, y, w, h) = cv2.boundingRect(contour)
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    if not motion_detected and motion_start_time is not None:
        motion_end_time = hora_atual
        motion_duration = motion_end_time - motion_start_time
        print(f"Movimento Terminado. Duração: {motion_duration}")
```

Figura 3 - Algoritmo *Python* para a deteção de movimento.

A variável *hora_atual* (Figura 4) foi definida através da função *datetime.now()* que está integrada no módulo *datetime*, utilizada para obter um objeto *datetime* que representa a data e hora atuais no momento em que a função é chamada [4], momento esse, definido inicialmente no próprio algoritmo (Figura 5) e, posteriormente, recebido através dos dados registados na base de dados (Figura 6).

```
# Obter a hora atual
hora_atual = datetime.now()
```

Figura 4 – Definição da variável *hora_atual* do tipo *datetime*.

```
tempo_adormecer = datetime.now().replace(hour=15, minute=22, second=00, microsecond=0)
tempo_acordar = datetime.now().replace(hour=15, minute=23, second=45, microsecond=0)
```

Figura 5 – Definição dos horários iniciais e finais de sono antes da criação da base de dados.

```
hora_inicial_formatada, hora_final_formatada, username = dados_to_python()
```

Figura 6 – Definição dos horários iniciais e finais de sono depois da criação da base de dados.

As variáveis *tempo_adormecer* e *tempo_acordar* (Figura 5) correspondem, respetivamente, às variáveis *hora_inicial_formatada* e *hora_final_formatada* (Figura 6), antes e depois da criação da base de dados.

Foi ainda necessário aceder à variável *username* proveniente do *website* para fazer corresponder ao utilizador (Figura 6).

O tempo total em que o utilizador permanece acordado é então calculado através da soma de todos intervalos de tempo em que se detetou movimento (Figura 7).

```
# Atualizar o tempo acordado total
tempo_acordado_total += motion_duration
```

Figura 7 – Cálculo do tempo de vigília total.

O tempo de sono é, finalmente, calculado, caso o sono ocorra durante o dia (o que implica que o horário final é "maior" que o inicial), através da subtração do horário para acordar pelo horário para adormecer, ambos definidos pelo utilizador, e ainda pelo tempo em que se registou movimento, em que se assume que o utilizador se encontra acordado (Figura 8).

```
# Calcular tempo a dormir em segundos
tempo_a_dormir = (tempo_acordar_segundos - tempo_adormecer_segundos) - tempo_acordado_total.total_seconds()
```

Figura 8 – Cálculo do tempo de sono total.

No caso em que o sono do utilizador decorre durante a noite, tem de se calcular o tempo decorrido desde a hora inicial escolhida até à meia-noite, e somar com o tempo decorrido desde a meia-noite até à hora final escolhida (para acordar, por exemplo, de manhã) (Figura 9).

```
if hora_inicial_formatada > hora_final_formatada:

    # Calcular o tempo até meia-noite
    tempo_ate_meia_noite = datetime.combine(data_atual, datetime.min.time()) + timedelta(days=1) - tempo_adormecer

    # Calcular o tempo desde meia-noite até o horário final
    tempo_desde_meia_noite = datetime.combine(data_atual, hora_final_formatada) - datetime.combine(data_atual, datetime.min.time())

    # Soma dos tempos
    tempo_entre_horas = tempo_desde_meia_noite + tempo_ate_meia_noite
```

Figura 9 – Cálculo do tempo de sono total (durante a noite).

Numa fase inicial de testes, foram criados ficheiros de texto com os dados que seriam posteriormente enviados para a base de dados depois de esta ser criada (Figura 10).

```
# Registar os tempos no arquivo de texto
with open("tempos_registados.txt", "w") as f:
    f.write(f"Tempo acordado durante a noite: {tempo_acordado_formatado}\n")
    f.write(f"Tempo a dormir durante a noite: {tempo_a_dormir_formatado}\n")
```

Figura 10 – Registo de dados em ficheiros de texto.

Depois da criação da base de dados foi necessário retornar as variáveis que se pretendiam enviar para a mesma (Figura 11).

```
# # Retornar as variáveis
return tempo_a_dormir_total, tempo_acordado_total, username
```

Figura 11 – Retorno das variáveis que contém o tempo de sono e de vigília total, e *username*.

Numa fase posterior, e, após a comunicação com o website, foi implementado o código da Figura 12, que permitiu a interrupção da deteção do movimento do código e o encerramento da câmara, automaticamente, no horário selecionado para acordar.

```
# Verificar se é hora de desligar a câmera
if cv2.waitKey(30) == ord('q') or hora_atual.time() >= tempo_acordar:
    break
```

Figura 12 – Código implementado para o encerramento da câmara.

2.1.2 Módulo *python_send_to_mysql.py*

Este algoritmo teve como objetivo enviar os dados do *Python* para a base de dados, neste caso, os tempos registados de sono e vigília, e respetivo *username*, para a tabela "registo_sono".

Para enviar os dados para a base foi necessário importar a função *obter_tempos()* definida no módulo *detecao_mov.py*, referido em 2.1.1, de forma a obter os tempos de sono e vigília (Figura 13).

```
import mysql.connector
from datetime import datetime

from detecao_mov7 import obter_tempos

# Chamar a função para obter os tempos
tempo_a_dormir_total, tempo_acordado_total, username = obter_tempos()
```

Figura 13 – Módulo *mysql.connector* e chamada da função *obter_tempos()*.

Para conectar a base de dados *MySQL* com *Python* foi importado o módulo *mysql.connector* (Figura 13) [5], e executar o código que se encontra na Figura 14, sabendo que a base de dados é local.

```
conexao = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="dreamtrack"
)

# Criar um cursor para interagir com o banco de dados
cursor = conexao.cursor()
```

Figura 14 – Código *Python* que permite a conexão do *Python* com a base de dados.

Por fim, para inserir os dados na tabela da base de dados, foi utilizado o código que se encontra na Figura 15. Os tempos registados foram enviados em formato *string* para serem compatíveis com os dados definidos na tabela "registo_sono", bem como, o *username* para associar ao utilizador e a data do registo.

```
inserir_dados_query = "INSERT INTO registodosono (tempo_acordado, tempo_sono, data, username) VALUES (%s, %s, %s, %s)"
valores = (tempo_acordado_total, tempo_a_dormir_total, data, username)


cursor.execute(inserir_dados_query, valores)

# Commit para salvar as alterações
conexao.commit()

conexao.close()
```

Figura 15 – Código *Python* para a inserção de valores na tabela "registo_sono" da base de dados *dreamtrack*.

2.1.3 Módulo *mySQL_to_python.py*

Este algoritmo teve como objetivo receber os dados da base de dados para o *Python*, neste caso, o horário de sono inicial e final registados na tabela "horário", para iniciar e parar o registo do tempo de acordo com os mesmos.

Depois de implementar novamente o código que se encontra na Figura 14, executa-se o que se encontra na figura seguinte (Figura 16).

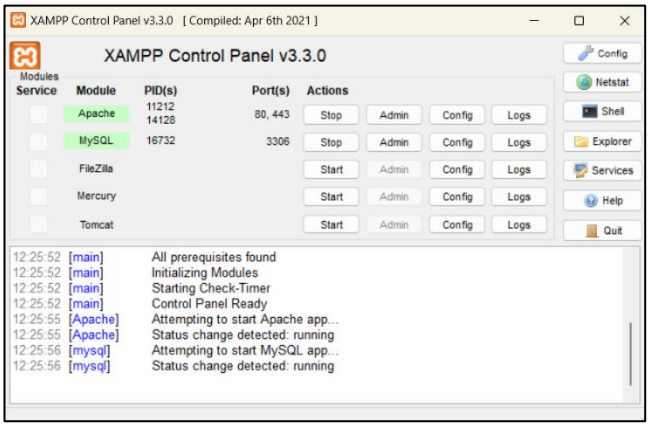
```
# Executar a consulta SQL
cursor.execute('SELECT adormecer, acordar FROM horário')

# Buscar todos os resultados da consulta
dados = cursor.fetchall()

# Processar e retornar os resultados como objetos datetime
for linha in dados:
    hora_inicial, hora_final = linha

    # Tratar a hora inicial se necessário
    if isinstance(hora_inicial, timedelta):
        hora_inicial_formatada = (datetime(1, 1, 1) + hora_inicial).time()
    else:
        hora_inicial_formatada = datetime.strptime(hora_inicial, "%H:%M:%S").time()

    # Tratar a hora final se necessário
    if isinstance(hora_final, timedelta):
        hora_final_formatada = (datetime(1, 1, 1) + hora_final).time()
    else:
        hora_final_formatada = datetime.strptime(hora_final, "%H:%M:%S").time()

# Fechar a conexão
conexao.close()

return hora_inicial_formatada, hora_final_formatada
```

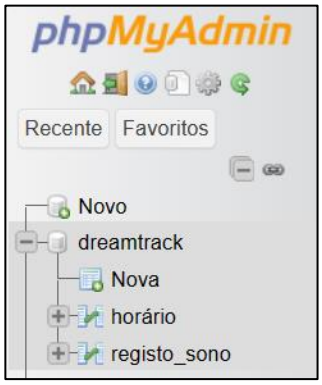
Figura 16 – Código *Python* para a seleção de valores da tabela "horário" da base de dados *dreamtrack*.

Os valores das tabelas da base de dados são convertidos para tipo *time* e retornados para poderem ser feitos os cálculos no módulo *detecao_mov.py*.

2.2 Criação de base de dados

Através da inicialização do *software* XAMPP e ativação dos serviços *Apache* e *MySQL* (Figura 17) necessários para aceder à interface do *phpMyAdmin* (Figura 18) [6], foi possível a criação da base de dados *dreamtrack* com duas tabelas: uma denominada "horário" onde foram inseridos os horários para adormecer e para acordar (que seriam enviados para a base de dados através do *website* por escolha dos mesmos pelo utilizador), que foram posteriormente enviados para o *Python* de forma a que o programa desenvolvido inicie e pare o registo de movimento de acordo com os mesmos; e uma segunda tabela com o nome "registodosono" que contém o registo dos dados calculados pelo algoritmo.





Figuras 17 e 18 – XAMPP *Control Panel;* Base de dados *dreamtrack* com as tabelas "horário" e "registo_sono".

2.3 Criação do histórico

A representação gráfica do histórico de sono foi realizada em *php*, sendo para isso, necessário o download do *software Visual Code Studio (VSCode* [7]*)*. Foram implementados dois *scripts*: um que faz a conexão com a base de dados e envia para a mesma os gráficos pretendidos (*historico.php*), e, outro (*ObterDados.php*) que permite selecionar os dados que irão aparecer no gráfico, de acordo com a escolha feita pelo utilizador (Figura 19).

```php
<?php

function obterDadosParaGrafico($username, $month, $dia_inicial, $dia_final) {
    global $conn; // Certifique-se de que $conn está disponível neste ponto

    $query = "SELECT DAY(data) as dia, AVG(tempo_sono) as media_tempo_dormir
              FROM registodosono
              WHERE MONTH(data) = $month AND DAY(data) BETWEEN $dia_inicial AND $dia_final
              AND username = '$username'
              GROUP BY dia";

    $result = $conn->query($query);

    $dataPoints = array();
    while ($row = $result->fetch_assoc()) {
        $dataPoints[] = array("label" => "Dia " . $row["dia"], "y" => $row["media_tempo_dormir"]);
    }

    return $dataPoints;
}

?>
```

Figura 19 – Implementação da função *obterDadosParaGrafico* do *script obterDados.php*.

Foi feita a conexão com a base de dados *dreamtrack* com o código representado na Figura 20.

```html
<!DOCTYPE HTML>
<html>
<head>
    <script src="https://cdn.canvasjs.com/canvasjs.min.js"></script>
</head>
<body>

<?php
$servername = "127.0.0.1";
$username = "root";
$password = "";
$dbname = "dreamtrack";

// Cria conexão
$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
```

Figura 20 – Código *php* implementado para conexão com a base de dados *dreamtrack*.

Para a criação do gráfico foi implementado o código da Figura 21. O gráfico representado é um gráfico de barras em que no eixo das ordenadas está representado o tempo (em segundos para permitir a realização de testes) e no eixo das abcissas o intervalo de dias escolhido pelo utilizador.

```
<div id="chartContainer" style="height: 370px; width: 100%;"></div>


<script>
window.onload = function () {
    CanvasJS.addColorSet("blueShades",
                [
                "#007FBF",
                "#00547F",
                "#002A40",
                "#0087CC",
                "#006EA6"
                ]);

    var chart = new CanvasJS.Chart("chartContainer", {
        animationEnabled: true,
        exportEnabled: true,
        colorSet:  "blueShades",
        title: {
            text: "Dias:  <?php echo $dia_inicial; ?> - <?php echo $dia_final; ?>       mes: <?php echo $nome_mes; ?>"
        },
        axisY: {
            title: "Tempo de Sono (segundos)"
        },

        data: [{
            type: "column",
            dataPoints: <?php echo json_encode($dataPoints, JSON_NUMERIC_CHECK); ?>
        }]
    });

    chart.render();
}
</script>
```

Figura 21 – Código *php* implementado para criação dos gráficos de barras.

2.4 Visão geral

O esquema seguinte (Figura 22) representa a visão geral da parte desenvolvida relativamente à visão computacional no projeto.

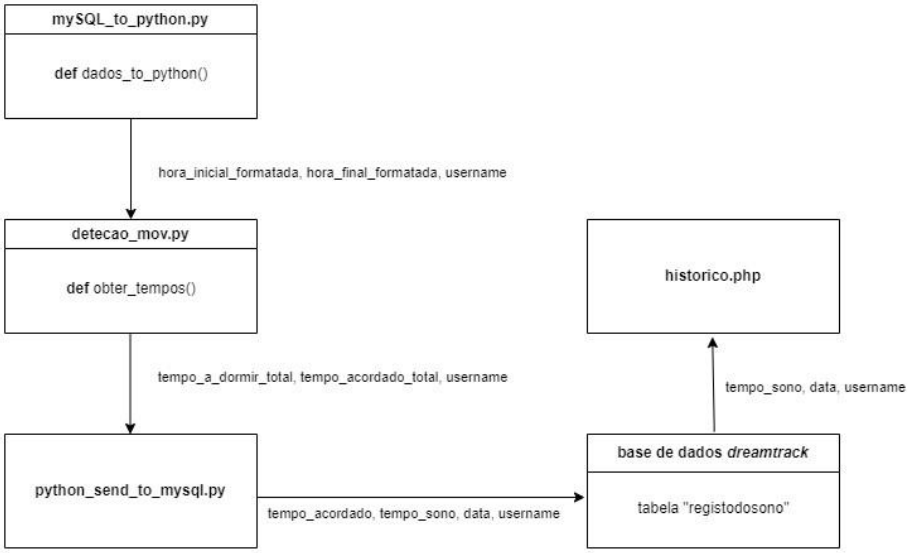


Figura 22 – Esquema da visão geral da visão computacional e registo.

Como podemos observar as variáveis *hora_inicial_formatada, hora_final_formatada, username*, provenientes da base de dados, escolhidas previamente pelo utilizador no website, são chamadas na função *obter_tempos()* de modo a obter as variáveis: *tempo_a_dormir_total, tempo_acordado_total, username*, que serão convertidas no ficheiro *python_send_to_mysql* a fim de serem enviadas para a base

de dados, e desta, para o *script php* de modo a representar graficamente o tempo de sono do utilizador durante um certo intervalo de dias.

3. RESULTADOS OBTIDOS

Para facilitar a realização de testes, estabeleceu-se um intervalo de tempo de minutos/segundos entre os horários definidos para o início e fim do sono (Figura 23).

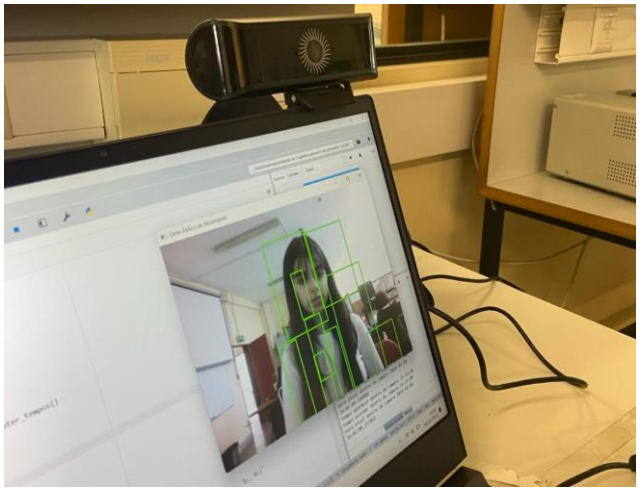


Figura 23 – Deteção de movimento através de algoritmos *Python* e câmara utilizada.

Nesta secção será apresentado o teste sublinhado na Tabela 1, para o *username* "Cristina12", no qual o horário definido para o início do sono foi às 12:57:00 e o horário de fim às 10:58:45, ou seja, 1 minuto e 45 segundos de intervalo, como se pode observar na tabela "horário" representada na Tabela 1. Estes horários foram então enviados, através do módulo *Python mySQL_to_python.py*, para o módulo principal, que determinou os tempos necessários enviados para a tabela "registo_sono" (Tabela 2) através do módulo *python_to_mySQL.py*, tendo-se obtido os valores de 32 segundos e 1 minuto e 12 segundos para os tempos de vigília e de sono, respetivamente, a data da realização e o *username* do utilizador para tornar possível a criação do gráfico em *php*.

Tabelas 1 e 2 – Tabelas da base de dados *dreamtrack: a)* "horário" inserida manualmente; b) "registo_sono" registada *através dos cálculos efetuados em Python*.

adormecer	acordar	username
10:44:00	10:45:00	Cristina12
10:46:00	10:47:00	Cristina12
10:50:00	10:51:30	Cristina12
10:53:00	10:54:00	AnaSofia
23:59:30	00:00:30	
23:59:30	00:00:30	
23:59:30	00:00:30	AnaSofia
12:03:00	12:04:00	Cristina12
12:10:00	12:11:30	Cristina12
12:50:00	12:51:00	Cristina12
12:57:00	12:58:45	Cristina12
22:20:00	07:30:00	AnaSofia

tempo_acordado	tempo_sono	data	username
00:00:28	00:01:01	2024-01-31	Cristina12
00:00:19	00:00:40	2024-01-31	AnaSofia
00:00:30	00:00:30	2024-01-24	AnaSofia
00:00:25	00:00:54	2024-01-24	Cristina12
00:00:10	00:00:50	2024-01-25	Cristina12
00:00:45	00:00:21	2024-01-26	AnaSofia
00:00:26	00:00:25	2024-01-28	Cristina12
00:00:00	-00:00:01	2024-01-31	AnaSofia
00:00:00	-00:00:01	2024-01-31	AnaSofia
00:00:00	-00:00:01	2024-01-31	AnaSofia
00:00:00	00:00:00	2024-01-31	AnaSofia
00:00:00	00:00:00	2024-01-31	AnaSofia
00:00:00	-00:00:01	2024-01-31	AnaSofia
00:00:00	-00:00:01	2024-01-31	AnaSofia
00:00:00	-00:00:01	2024-01-31	AnaSofia
00:00:00	00:01:00	2024-01-31	AnaSofia
00:00:00	00:00:59	2024-01-31	Cristina12
00:00:00	00:00:59	2024-01-31	Cristina12
00:00:00	00:01:29	2024-01-31	Cristina12
00:00:49	00:00:10	2024-01-31	Cristina12
00:00:32	00:01:12	2024-01-31	Cristina12
00:00:00	09:10:00	2024-01-31	AnaSofia
00:00:50	00:00:55	2024-01-22	Cristina12
00:00:25	00:00:54	2024-01-23	Cristina12

Desta forma, foi criado o histórico para o *username* “Cristina12”, como podemos observar na Figura 24, para os dias entre 20 e 31 de janeiro.

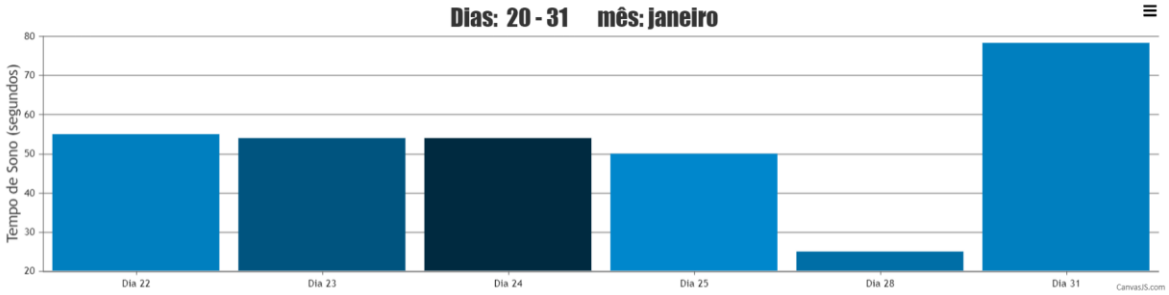


Figura 24 – Histórico de sono representado graficamente para o *username* “Cristina12”.

4. CONCLUSÃO

Sendo os objetivos deste módulo do projeto a implementação de algoritmos *Python* que permitissem o registo e cálculo dos tempos de sono e de vigília do utilizador durante a noite, bem como, a implementação de código *php* que permitisse a visualização de um gráfico de barras que descrevesse o tempo de sono do utilizador ao longo de um intervalo de dias, pode-se então afirmar que estes foram concluídos com sucesso.

No entanto, é importante notar que esta parte poderá não apresentar uma fiabilidade a cem porcento, uma vez que, a função de visão computacional utilizada ainda apresenta limitações, podendo não ser totalmente precisa. Para verificar a viabilidade do algoritmo deveriam ter sido efetuados testes durante intervalos de tempo maiores, por exemplo, durante a noite, o que não foi possível devido à inexistência de uma câmara com visão noturna, pelo que, com uma câmara normal, nesta situação, não seria detetado qualquer tipo de movimento.

Em suma, e apesar das limitações envolvidas, o desenvolvimento desta parte do projeto permitiu o aprimoramento das minhas capacidades técnicas e de resolução de problemas, tendo proporcionado um aumento de autonomia e resiliência perante a complexidade dos desafios que surgiram ao longo da realização do mesmo. É de notar que o sucesso na implementação dos algoritmos representa um avanço significativo na concretização dos objetivos deste projeto.

5. REFERÊNCIAS

[1] Simplilearn, “What Is Computer Vision: Applications, Benefits and How to Learn It,” 10 agosto 2023. [Online]. Available: https://www.simplilearn.com/computer-vision-article#what_is_computer_vision.

[2] GeeksforGeeks, “OpenCV Python Tutorial,” 2023. [Online]. Available: https://www.geeksforgeeks.org/opencv-python-tutorial/.

[3] GeeksforGeeks, “Python OpenCV – Background Subtraction,” 2023. [Online]. Available: https://www.geeksforgeeks.org/python-opencv-background-subtraction/.

[4] Python Documentation, “datetime — Basic date and time types,” Python Software Foundation, 2023. [Online]. Available: https://docs.python.org/3/library/datetime.html.

[5] W3Schools, “Python MySQL,” 2024. [Online]. Available: https://www.w3schools.com/python/python_mysql_getstarted.asp.

[6] K. &. K. R. &. K. A. &. R. J. &. S. A. &. D. A. Joshi, “A Framework Optimization in Social Media using Xampp: A Systematic Approach,” 2022.

[7] “Visual Studio Code,” 2024. [Online]. Available: https://code.visualstudio.com/api/references/vscode-api.